Gradient Descent

1. What Is Gradient Descent?

Gradient descent is an iterative optimization algorithm used to minimize a differentiable loss function J(θ). **At each step, the parameters are moved in the opposite direction of the gradient**, because that is the direction in which the loss decreases most quickly.

- Training neural networks
- Linear regression and logistic regression
- Many other machine-learning models that rely on minimizing a loss function

Intuition

Imagine standing blindfolded on a hilly landscape. You cannot see the lowest point, but you can feel the slope under your feet.
If you always step a little in the steepest downhill direction, you gradually move toward a minimum. That is exactly the intuition behind gradient descent.

2. Mathematical Foundation

For a loss function J(θ), the gradient ∇J(θ) points in the direction of steepest increase. Therefore, to move downhill we update the parameter in the opposite direction.

$$\theta_{new} = \theta_{old} - \alpha \times \nabla J(\theta_{old})$$

- α (alpha): learning rate — a small positive value that controls step size.
- ∇J(θ): gradient of the loss with respect to the parameter(s).
- In one dimension, the gradient becomes the ordinary derivative dJ/dθ.

Algorithm 1 — Batch Gradient Descent (Pseudocode)

```
initialize θ with a guess
while not converged:
   compute gradient g = ∇J(θ)
   θ ← θ − α × g
return θ
```

Code Snippet — Python-style Representation

```
theta = initial_value
for _ in range(max_iters):
   grad = grad_J(theta)
   theta = theta - alpha * grad
return theta
```

3. Worked Example: Minimizing $J(\theta) = \theta^2$

We will use the one-dimensional loss function $J(\theta) = \theta^2$. The true minimum occurs at $\theta = 0$, and the gradient is $dJ/d\theta = 2\theta$.

Initial value	Gradient	Goal
$\theta^{(0)} = 5$	$dJ/d\theta = 2\theta$	Observe how α affects convergence

Case A — $\alpha = 0.1$ (good learning rate)

Iter.	Current θ	Gradient	Updated θ	Loss J(updated θ)
1	5	10	4	16
2	4	8	3.2	10.24
3	3.2	6.4	2.56	6.5536
4	2.56	5.12	2.048	4.1943

Interpretation: The parameter moves steadily toward 0 and the loss decreases quickly without instability. This is the behavior we usually want.

Case B — $\alpha = 0.01$ (too small)

Iter.	Current θ	Gradient	Updated θ	Loss J(updated θ)
1	5	10	4.9	24.01
2	4.9	9.8	4.802	23.0592
3	4.802	9.604	4.70596	22.1461

Interpretation: The updates are safe but very slow. The optimizer is moving in the right direction, yet it needs many iterations to make visible progress.

Case C — $\alpha = 0.6$ (large; oscillatory but still convergent here)

Iter.	Current θ	Gradient	Updated θ	Loss J(updated θ)
1	5	10	−1	1
2	−1	−2	0.2	0.04
3	0.2	0.4	−0.04	0.0016
4	−0.04	−0.08	0.008	0.000064

Interpretation: For $J(\theta)=\theta^2$, $\alpha = 0.6$ makes the updates bounce around 0, but the oscillations shrink and the method still converges.

Learning-rate summary

Learning Rate α	Convergence Speed	Stability	Outcome
Very small (e.g. 0.001)	Extremely slow	Very Stable	Takes thousands of iterations
Good (e.g. 0.01 – 0.1)	Fast & Smooth	Stable	Reaches minimum efficiently
Large (e.g. 0.5-1.0)	Very fast at first	Oscillates	May oscillate around minimum
Too large (>1.0)	-	Diverges	Loss explodes → algorithm fails

Rule of Thumb

Start with a moderate learning rate such as 0.01 to 0.1, observe training behavior, and then adjust.
Modern optimizers such as Adam adapt the step size automatically, but the basic gradient-descent idea remains the same.

4. When the Function Has Multiple Parameters

The same principle applies when there are many parameters. Each parameter is updated using its own partial derivative.

$$[\ \theta_1_new\ ] = [\ \theta_1_old\ ] - \alpha\ [\ \partial J/\partial\theta_1\ ]$$
$$[\ \theta_2_new\ ] = [\ \theta_2_old\ ] - \alpha\ [\ \partial J/\partial\theta_2\ ]$$

So, even in higher dimensions, gradient descent still follows one simple rule: move every parameter a little against the gradient.

5. Practice Questions

Question 1:

Loss function: $J(\theta) = (\theta - 3)^2$
Initial value: $\theta = 0$, learning rate: $\alpha = 0.2$
Task: Compute θ after exactly 3 iterations. Show every step.

Question 2:

Use the same loss $J(\theta) = (\theta - 3)^2$ with initial $\theta = 0$.
a) Use $\alpha = 0.05$ and compute θ after 2 iterations.
b) Use $\alpha = 1.0$ and compute θ after 2 iterations.
c) Which setting converges faster? Explain briefly.

Question 3:

Loss function: $J(\theta) = (2\theta - 5)^2$
Initial value: $\theta = 1$, learning rate: $\alpha = 0.1$
Task: Find θ after 4 iterations.

Question 4:

In 2–3 sentences, explain what happens when the learning rate is too large.
Then sketch the parameter path for $J(\theta) = \theta^2$ on paper.

Question 5:

Loss function: $J(\theta) = \theta^4$
Gradient: $4\theta^3$
Initial value: $\theta = 2$, learning rate: $\alpha = 0.05$
Task: Derive the update rule and compute the first update.